

Программная реализация симплекс-метода

В программе самостоятельно реализованы выбор разрешающего столбца и строки, пересчёт симплекс-таблицы, решение вспомогательной задачи, переход к основной задаче и получение итогового решения.

Выбор разрешающего столбца и строки

```
def find_entering_column(table):  
    """Выбор разрешающего столбца"""  
  
    objective_row = table[-1][: -1]  
    min_value = min(objective_row)  
  
    if min_value >= 0:  
        return None  
  
    return objective_row.index(min_value)  
  
def find_leaving_row(table, column):  
    """Выбор разрешающей строки"""  
  
    best_row = None  
    best_ratio = None  
  
    for i in range(len(table) - 1):  
        a = table[i][column]  
        b = table[i][ -1]  
  
        if a > 0:  
            ratio = b / a  
  
            if best_ratio is None or ratio < best_ratio:  
                best_ratio = ratio  
                best_row = i  
  
    return best_row
```

После выбора разрешающего столбца определяется переменная, которая должна выйти из базиса. Для положительных элементов выбранного столбца вычисляется отношение свободного члена к соответствующему коэффициенту:

$$b_i/a_{ij}$$

Из полученных значений выбирается минимальное.

После выбора разрешающей строки и столбца выполняется пересчёт симплекс-таблицы.

Сначала пересчитывается разрешающая строка. Все её элементы делятся на разрешающий элемент. Для элемента, находящегося на пересечении разрешающей строки и столбца, используется обратное значение разрешающего элемента.

Остальные строки пересчитываются по формулам симплекс-метода.

```
def pivot(table, basis, nonbasis, row, column):
    pivot_element = table[row][column]

    old_pivot_row = table[row][:]

    leaving_variable = basis[row]
    entering_variable = nonbasis[column]

    print(
        f"\nБазис входит {entering_variable}, "
        f"из базиса выходит {leaving_variable}."
    )
    print(f"Ведущий элемент = {pivot_element}")

    # Пересчитываем ведущую строку
    new_pivot_row = [f(0) for _ in table[row]]

    for j in range(len(nonbasis)):
        if j == column:
            new_pivot_row[j] = f(1) / pivot_element
        else:
            new_pivot_row[j] = old_pivot_row[j] / pivot_element

    new_pivot_row[-1] = old_pivot_row[-1] / pivot_element

    # Пересчет остальных строк
    for i in range(len(table)):
        if i == row:
            continue

        old_row = table[i][:]
        coefficient = old_row[column]

        for j in range(len(nonbasis)):
```

```

        if j == column:
            table[i][j] = -coefficient / pivot_element
        else:
            table[i][j] = (
                old_row[j]
                - coefficient * old_pivot_row[j] / pivot_element
            )

    table[i][-1] = (
        old_row[-1]
        - coefficient * old_pivot_row[-1] / pivot_element
    )

    table[row] = new_pivot_row

    basis[row] = entering_variable
    nonbasis[column] = leaving_variable

```

Для автоматического выполнения всех итераций используется функция `simplex()`. Она повторяет выбор разрешающего столбца и строки и выполняет пересчёт таблицы до тех пор, пока в последней строке не останется отрицательных коэффициентов.

```

def simplex(table, basis, nonbasis, phase_name):
    """Симплекс-шаги"""
    step = 1
    while True:
        column = find_entering_column(table)

        if column is None:
            print(f"\n{phase_name}: отрицательных коэффициентов больше нет.")
            break

        row = find_leaving_row(table, column)
        if row is None:
            raise ValueError("Целевая функция не ограничена снизу.")
        pivot(table, basis, nonbasis, row, column)
        print_table(
            table,
            basis,
            nonbasis,
            f"{phase_name}. Таблица после шага {step}",
        )
        step += 1

```

Построение начальной таблицы вспомогательной задачи

На вход функции передаются коэффициенты целевой функции и ограничения исходной задачи.

Сначала формируется список исходных переменных $x_1 \dots x_4$. Затем для ограничений со знаками добавляются дополнительные переменные. После этого формируются строки ограничений и создаются искусственные переменные, которые образуют начальный базис вспомогательной задачи.

```
def build_phase_one(c, constraints):
    """начальная таблица"""

    number_of_variables = len(c)

    original_variables = [
        f"x{i}"
        for i in range(1, number_of_variables + 1)
    ]

    next_variable_number = number_of_variables + 1

    additional_variables = []
    additional_for_row = {}

    # добавляем дополнительные переменные:

    for i, (_, sign, _) in enumerate(constraints):

        if sign == "<=":
            name = f"x{next_variable_number}"
            next_variable_number += 1

            additional_variables.append(name)
            additional_for_row[i] = (name, f(1))

        elif sign == ">=":
            name = f"x{next_variable_number}"
            next_variable_number += 1

            additional_variables.append(name)
            additional_for_row[i] = (name, f(-1))

    nonbasis = original_variables + additional_variables

    variable_index = {
        name: i
        for i, name in enumerate(nonbasis)
```

```

}

rows = []

# Строим строки ограничений
for i, (coefficients, sign, b) in enumerate(constraints):

    row = [f(value) for value in coefficients]

    row += [f(0) for _ in additional_variables]

    if i in additional_for_row:
        name, value = additional_for_row[i]
        row[variable_index[name]] = value

    row.append(f(b))
    rows.append(row)

# Создаем искусственные переменные

artificial = set()
basis = []

for _ in constraints:
    name = f"x{next_variable_number}"
    next_variable_number += 1

    artificial.add(name)
    basis.append(name)

# Вспомогательная целевая функция
objective_row = [
    f(0)
    for _ in range(len(nonbasis) + 1)
]

for row in rows:
    for j in range(len(row)):
        objective_row[j] -= row[j]

table = rows + [objective_row]

return table, basis, nonbasis, artifici

```

Последняя строка таблицы соответствует вспомогательной целевой функции. Так как искусственные переменные изначально входят в базис, строка вспомогательной функции формируется как отрицательная сумма строк ограничений. В результате программа получает ту же начальную симплекс-таблицу, которая была получена при ручном решении.

После завершения первой фазы необходимо убедиться, что искусственные переменные не остались в базисе. Даже если минимум вспомогательной функции равен нулю, искусственная переменная теоретически может оставаться базисной со значением 0.

Функция `remove_artificial_from_basis()` проверяет базис. Если в нём остаётся искусственная переменная с нулевым значением, программа ищет обычную небазисную переменную, которую можно ввести вместо неё.

```
def remove_artificial_from_basis(table, basis, nonbasis, artificial):
    i = 0
    while i < len(basis):
        if basis[i] not in artificial:
            i += 1
            continue

        if table[i][-1] != 0:
            raise ValueError(
                "Искусственная переменная осталась "
                "в базисе с ненулевым значением.")

        pivot_column = None

        for j, variable in enumerate(nonbasis):

            if (
                variable not in artificial
                and table[i][j] != 0
            ):
                pivot_column = j
                break

        if pivot_column is not None:

            pivot(
                table,
                basis,
                nonbasis,
                i,
                pivot_column
            )
```

```

        i += 1

    else:
        del table[i]
        del basis[i]

```

После удаления искусственных переменных выполняется переход к исходной задаче. Для этого необходимо восстановить строку исходной целевой функции. Функция `build_original_objective()` задаёт коэффициенты исходных переменных, а для дополнительных переменных принимает коэффициент равным нулю. Затем для каждой небазисной переменной вычисляется новая оценка с учётом текущего базиса.

```

def build_original_objective(table, basis, nonbasis, c):
    """Построение строки исходной целевой функции."""

    cost = {}

    for i, value in enumerate(c):
        cost[f"x{i + 1}"] = f(value)

    for variable in basis + nonbasis:
        if variable not in cost:
            cost[variable] = f(0)

    objective_row = []

    # Для каждого небазисного элемента считаем коэффициент
    for j, variable in enumerate(nonbasis):

        coefficient = cost[variable]

        for i, basic_variable in enumerate(basis):

            coefficient -= (
                cost[basic_variable]
                * table[i][j]
            )

        objective_row.append(coefficient)

    # Свободный член целевой функции:
    objective_value = f(0)

    for i, basic_variable in enumerate(basis):

        objective_value += (

```

```

        cost[basic_variable]
        * table[i][-1]
    )
    objective_row.append(-objective_value)
    table.append(objective_row)
    return table

```

После формирования этой строки начинается вторая фаза симплекс-метода.

Исходная задача задаётся в программе непосредственно через коэффициенты целевой функции и ограничения.

Каждое ограничение хранится в виде тройки: коэффициенты левой части, знак ограничения и правая часть.

```

# Z = 4*x1 + x2 + x3 + 2*x4 -> min
# Коэффициенты целевой функции
c = [4, 1, 1, 2]

# Ограничения
constraints = [
    ([2, 1, 0, 1], "<=", 9),
    ([1, 1, 1, 0], "=", 7),
    ([0, 0, 1, 1], ">=", 5),
]

table, basis, nonbasis, artificial = build_phase_one(c, constraints)
print_table(table, basis, nonbasis, "Начальная таблица вспомогательной задачи",)
simplex(table, basis, nonbasis, "1")

# Проверка ответа вспомогательной задачи
if table[-1][-1] != 0:
    raise ValueError("У исходной задачи нет допустимых решений.")

print("\nМинимум вспомогательной функции равен 0.")
print("Значит, допустимые решения исходной задачи существуют.")

remove_artificial_from_basis(table, basis, nonbasis, artificial)

# Удаляем столбцы искусственных переменных

keep_columns = [i for i, name in enumerate(nonbasis) if name not in artificial]
new_nonbasis = [nonbasis[i] for i in keep_columns]

```

```

new_table = []

for i in range(len(basis)):
    row = [table[i][j] for j in keep_columns]
    row.append(table[i][-1])
    new_table.append(row)

nonbasis = new_nonbasis

# Переход к основной задаче

table = new_table
table = build_original_objective(table, basis, nonbasis, c)

print_table(table, basis, nonbasis, "Первая таблица основной задачи",)
simplex(table, basis, nonbasis, "2")
solution = {
    "x1": f(0),
    "x2": f(0),
    "x3": f(0),
    "x4": f(0),
    "x5": f(0),
    "x6": f(0),
}

for i, variable in enumerate(basis):
    if variable in solution:
        solution[variable] = table[i][-1]

# Значения исходных переменных x1, x2, x3, x4.
x1 = solution["x1"]
x2 = solution["x2"]
x3 = solution["x3"]
x4 = solution["x4"]

# Значение исходной целевой функции
z = 4 * x1 + x2 + x3 + 2 * x4

print("ОТВЕТ")

print(f"x1 = {x1}")
print(f"x2 = {x2}")
print(f"x3 = {x3}")
print(f"x4 = {x4}")

```

```
print(f"Z_min = {z}")

print("\nВектор решения:")
print(f"x* = ({x1}; {x2}; {x3}; {x4})")
```

В результате программного решения получено:

```
ОТВЕТ
x1 = 0
x2 = 2
x3 = 5
x4 = 0
Z_min = 7

Вектор решения:
x* = (0; 2; 5; 0)
```

Следовательно, полученный результат совпадает с решением, найденным вручную симплекс-методом.